

Supplementary Material for ForeTac: Steering Robot Actions with Predicted Contact Consequences

Anonymous Authors

A Appendix Overview

This appendix expands the main paper with concrete architecture, collection, preprocessing, training, evaluation, analysis, and limitation details. Section B describes the real-robot data format and training data statistics. Section C describes the network components. Section D gives the model hyperparameters and training contracts. Section E gives the inference-time algorithm used by ForeTac. Sections F–I provide additional quantitative and qualitative evidence for the tactile representation, tactile foresight, contact-quality scorer, and force behavior. The remaining sections summarize the main comparison, ablations, insertion benchmark, backbone adaptation, failure cases, deployment overhead, and limitations.

B Data Collection and Format

B.1 Robot Observation Schema

All policies and foresight modules are trained from timestamp-aligned HDF5 episodes. A trajectory contains synchronized RGB views, proprioception, action labels, tactile marker fields, and optional force/torque measurements. The main action-conditioned foresight contract is

$$\hat{\mathbf{z}}_{t:t+H-1} = f_{\psi}(I_t^g, I_t^w, \mathbf{m}_{t-W+1:t}, q_t, \mathbf{a}_{t:t+H-1}), \quad (1)$$

where the condition is the future action chunk `actions/joint_abs[t:t+H]`, not the future state trajectory. This distinction is important: future states are outcomes, while the rollout-time quantity available before execution is the candidate action chunk.

RGB images are resized and cropped before ImageNet normalization. The tactile input is a GelSight marker displacement field, normalized per displacement channel. TacVAE consumes an eight-frame tactile window and encodes the current contact into a compact spatial latent. The diffusion policy uses `obs_horizon=2` and predicts a 16-step action chunk for the main experiments.

B.2 Collection Procedure

Each demonstration episode is recorded as a continuous robot execution trace. The controller saves either joint-absolute commands (`actions/joint_abs`) or end-effector commands (`actions/eef_abs / eef_rel`), depending on the task script. The final action-conditioned ForeTac experiments use joint-absolute chunks for board, card, and socket-style policies when those labels match the measured joint states. Vase data collected with older end-effector scripts is handled separately because some saved `joint_abs` labels contain inverse-kinematics branch jumps that are not present in the measured robot motion. For those sequences, end-effector labels or regenerated joint labels should be used for policy training.

The tactile stream is saved at the same episode step index as the RGB and robot state streams. The most important tactile field for this paper is `marker_offset`, a 9×9 grid of two-dimensional marker displacements. We prefer this marker field over raw tactile RGB for the foresee-and-score pipeline because it is compact, directly related to local contact deformation, and can be visualized as a deformed grid. Force/torque signals are used for labeling and analysis when available, but the main foresee-and-score chain does not require force sensing at deployment.

Table 1: HDF5 episode fields used by the implementation.

Field	Shape / type	Usage
observations/images/global	$T \times H \times W \times 3$ uint8	scene context
observations/images/wrist	$T \times H \times W \times 3$ uint8	local contact geometry
observations/proprio_joint	$T \times 7$ float	current robot state
actions/joint_abs	$T \times 7$ float	action chunk target / foresight condition
observations/tac/left/marker_offset	$T \times 9 \times 9 \times 2$ float	GelSight marker displacement
observations/tac/left/force6d	$T \times 6$ float	contact-quality labels and analysis

B.3 Training Data Statistics

Table 2 reports the tactile pretraining data scale used for the representation model. The corpus contains 5,969 HDF5 episodes and approximately 2.1 million tactile marker frames. The frame total follows the same marker-stream length convention used throughout the training pipeline.

Table 2: Tactile pretraining data scale.

Training corpus	Episodes	Approx. tactile frames
TacVAE pretraining corpus	5969	$\approx 2.1\text{M}$

B.4 TacVAE Pretraining Corpus

TacVAE is pretrained on the full tactile corpus rather than only on the five-task rollout split. The pretraining corpus contains 5,969 HDF5 episodes and approximately 2.1 million tactile marker frames. The corpus includes successful contact, bounce, too-small-force, too-large-force, oscillatory-force, insertion misalignment, and replayed trajectory variants. These negative or imperfect segments are useful for representation learning because the encoder must preserve both desirable and undesirable contact states; the contact-quality scorer later learns which regions of this tactile space should be preferred.

Task-local TacVAE checkpoints use the same architecture and normalization contract for board wiping, vase wiping, and card swiping. These task-local encoders are used when the deployment policy is task-specific; the full-corpus TacVAE is used as a broader tactile pretraining reference.

B.5 Task Labels and Safety Definitions

For each online trial, a success is counted when the task objective is completed within the rollout horizon. A safe success is stricter and is computed from the saved rollout log. Let $y_{\text{task}} \in \{0, 1\}$ denote task completion, F_t the contact-force magnitude, $\mathcal{C} = \{t : F_t > \tau_{\text{contact}}\}$ the contact frames, r the automatically detected retry or bounce count, and d_{obj} the measured or manually audited object displacement. We define

$$F_{\max} = \max_t F_t, \quad (2)$$

$$\sigma_F^2 = \text{Var}_{t \in \mathcal{C}}(F_t), \quad (3)$$

$$\rho_{\text{drop}} = 1 - \frac{|\mathcal{C}|}{T_{\text{expected}}}, \quad (4)$$

$$B_{\text{band}} = \frac{1}{|\mathcal{C}|} \sum_{t \in \mathcal{C}} \mathbb{I} \left[\tau_{\text{low}}^{(k)} \leq F_t \leq \tau_{\text{high}}^{(k)} \right]. \quad (5)$$

For task k , the safe-success label is

$$y_{\text{safe}} = y_{\text{task}} \mathbb{I}[F_{\text{max}} \leq \tau_{\text{peak}}^{(k)}] \mathbb{I}[\sigma_F^2 \leq \tau_{\text{var}}^{(k)}] \mathbb{I}[\rho_{\text{drop}} \leq \tau_{\text{drop}}^{(k)}] \cdot \mathbb{I}[B_{\text{band}} \geq \tau_{\text{band}}^{(k)}] \mathbb{I}[r \leq \tau_{\text{retry}}^{(k)}] \mathbb{I}[d_{\text{obj}} \leq \tau_{\text{obj}}^{(k)}] \mathbb{I}[\neg \text{damage}]. \quad (6)$$

For insertion, B_{band} can be omitted or replaced by a jamming-duration threshold. For wiping and swiping, the band-occupancy and variance terms are the main indicators of stable contact. Any visible object displacement, tip-over, grasped-object damage, emergency stop, or hardware safety intervention marks the trial as unsafe; severe object movement or damage is also counted as task failure even if the geometric endpoint is reached.

B.6 Task Naming and Vase Variants

The rollout tables use short English task names. “Board wiping” refers to blackboard erasing with a flat surface. “Vase side wiping” refers to wiping a curved vase side at a roughly fixed height. “Vase vertical wiping” is a separate task variant: the robot must maintain contact while moving across different heights of the curved object, covering lower, middle, and upper contact regions. This variant is reported separately because the same visual pose can correspond to different normal-force requirements as the local surface curvature and height change. “Card swiping” refers to pushing or sliding a card through a slot-like contact region, and “chip grasping” refers to lifting a fragile object without crush or slip.

C Network Architecture

C.1 Module Overview

ForeTac is a modular add-on to a chunk-based action generator. The full system contains four trainable or pretrained components:

1. a tactile encoder-decoder (TacVAE) that maps GelSight marker histories to compact latent maps and decodes them back to marker fields;
2. a base action policy, usually Diffusion Policy, that provides the behavioral prior over demonstrated action chunks;
3. an action-conditioned tactile foresight transformer that predicts the future tactile latent sequence caused by a candidate action chunk; and
4. a contact-quality energy model that maps the predicted tactile future to a scalar quality score and supplies gradients for action refinement.

This separation is intentional. The base policy is not asked to learn a new global objective at inference time; it only supplies candidate actions. The foresight model answers a counterfactual contact question for each candidate chunk. The scorer decides whether the predicted contact is desirable for the task. The guidance module then applies a bounded local correction to the action sample.

C.2 TacVAE Tokens

TacVAE produces a spatial latent rather than a single unstructured vector. In the implementation, the latent has 16 channels and a 3×3 spatial layout. The 3×3 cells are treated as tactile tokens after a linear projection to the 512-D transformer hidden dimension. This keeps the foresight module aware of where contact occurs on the GelSight surface. The decoder is kept in the training and diagnostic loop so that latent prediction can be checked in marker space.

Table 3: Main architecture parameterization.

Module	Input	Internal representation	Output
TacVAE encoder	8 marker frames, $9 \times 9 \times 2$ each	$16 \times 3 \times 3$ spatial latent	144-D latent vector / 9 tactile tokens
TacVAE decoder	$16 \times 3 \times 3$ latent	deconvolutional marker decoder	$9 \times 9 \times 2$ marker displacement
Vision encoder	global and wrist RGB	frozen ResNet18 tokens, projected to 512-D	visual context tokens
Foresight transformer	visual tokens, tactile tokens, q_t , action chunk	3 layers, 8 heads, hidden 512, FFN 2048	future latent sequence, $H = 16$
Contact-quality scorer	predicted latent/marker sequence, optional action	task-quality classifier / energy head	expert-good margin score
Guidance module	noisy diffusion sample and score gradient	late-step normalized trust-region update	refined action chunk

C.3 Action-Conditioned Foresight Transformer

The foresight transformer receives the current observation tokens and the candidate action chunk. Its prediction horizon is set to the same 16-step window used by the policy chunk. The action sequence is embedded as the conditioning signal for future contact dynamics. The current proprioceptive state q_t is included as context, but the future proprioceptive trajectory is not used in the main model because it would leak an outcome rather than condition on an executable candidate command.

C.4 Contact-Quality Energy Model

The scorer is trained to separate expert or acceptable contact from task-specific failure modes. For board wiping, the failure modes include pressure too small, pressure too large, and unstable pressure. For socket insertion, the important negatives include side contact, bounce, retry-heavy motion, jamming, and excessive peak force. The deployed scalar score is the logit margin between the good-contact class and the log-sum-exp of bad-contact classes. This score is less saturated than a probability and therefore provides more useful gradients for late denoising.

C.5 Guidance Module

The guidance module runs only during selected late denoising or flow-matching steps. It uses the current clean-action estimate, predicts the tactile future, computes the quality margin, and differentiates the margin with respect to the action sample. The update is normalized, clipped, and checked in raw action units. In the main latent-only guidance path, the action features inside the scorer are detached so that the gradient follows the intended path

$$\mathbf{a} \rightarrow f_\psi(\mathbf{o}, \mathbf{a}) \rightarrow S.$$

This reduces the risk that the scorer learns an action-only shortcut that does not correspond to the predicted tactile consequence.

D Training Details

D.1 Model Components

Table 4 summarizes the implementation settings used in the experiments. These values match the local scripts and configs used for the project page and appendix evidence.

Table 4: Main training hyperparameters.

Component	Architecture / input	Optimization	Notes
TacVAE	8-frame $9 \times 9 \times 2$ marker windows; latent $16 \times 3 \times 3$	AdamW, lr 10^{-4} , batch 512, 150 epochs, stride 2 or 4	KL weight 10^{-6} ; direction loss weight 0.2
Visual/tactile DP	global+wrist RGB, TacVAE latent history, q_t ; action horizon 16	AdamW, lr 10^{-4} , batch 64, 1000 epochs for board	DDPM training steps 100; inference steps 100; action space <code>joint_abs</code>
Action-conditioned foresight	ResNet18 visual tokens, TacVAE tokens, q_t , action chunk $H = 16$	AdamW, lr 4×10^{-5} , weight decay 10^{-4}	3 transformer layers, 8 heads, hidden 512, FFN 2048
Contact-quality scorer	predicted latent sequence and optional action/force features	cross-entropy / margin objective	deployed score is expert-good margin against negative modes
Guided denoising	late DDPM steps only	normalized gradient step; trust-region clipping	action branch detached in latent-only path

D.2 TacVAE Pretraining Setup

The tactile representation is trained before the policy and before the foresight model. Each training sample is an eight-frame marker window from one GelSight side. The target is the marker displacement field in the current window, and the reconstruction loss is computed after decoding the latent back to marker space. Marker fields are normalized per displacement channel using statistics computed from the training split. The same normalization statistics are saved with the checkpoint and reused by policy training, foresight training, visualization, and runtime serving.

Table 5: TacVAE pretraining data and optimization settings. Episode and frame counts are reported for the full pretraining corpus.

TacVAE setting	Episodes	Approx. frames	Window / stride	Optimization
Full tactile pretraining corpus	5969	$\approx 2.1\text{M}$	8 / mixed	AdamW, lr 10^{-4} , batch 512
Board task-local TacVAE			8 / 4	150 epochs, KL 10^{-6} , dir. 0.2
Vase task-local TacVAE			8 / 2	150 epochs, KL 10^{-6} , dir. 0.2
Card task-local TacVAE			8 / 2	150 epochs, KL 10^{-6} , dir. 0.2

The full-corpus number is used to describe the available tactile pretraining scale. Task-local encoders use task-specific marker statistics and contact patterns for board wiping, vase wiping, and card swiping.

D.3 Policy Training Setup

The base DP policy is trained in a standard denoising-action framework. The input condition is the concatenation of visual features, tactile latent history, and normalized robot state over a short observation horizon. The action target is a 16-step chunk in normalized `joint_abs` action space. For board wiping, the policy uses global and wrist RGB views, `obs_horizon=2`, `pred_horizon=16`, `n_action_steps=8`, batch size 64, 100 diffusion training steps, and 100 inference denoising steps. The model is trained for up to 1000 epochs with validation-window monitoring and periodic best-checkpoint saving.

For tasks where the saved joint action labels are known to contain inverse kinematics branch jumps, the policy should use end-effector action labels or regenerated joint labels. This is especially relevant for older vase data, where mixing a correct tactile model with an incorrect action label space would make the foresight conditioning appear worse than it is.

D.4 Scorer Training Setup

The contact-quality scorer is trained from labeled contact windows rather than from final task success alone. This gives dense supervision for contact quality: expert contact, insufficient pressure, excessive pressure, oscillatory contact, bounce, edge contact, jamming, and other task-specific negative modes. For the board setting, the scorer has a held-out validation accuracy and macro-F1 of 1.0 on the labeled validation windows used by the guidance diagnostic. This result evaluates offline contact labels rather than online task success.

At deployment, the scorer output is converted to an expert-margin energy. The margin is preferred over a calibrated probability because it remains useful when the classifier is confident. The same score can be used in two ways: candidate reranking after sampling, or differentiable guidance during the final denoising steps. The main ForeTac configuration uses the latter because it can locally improve a sampled chunk without discarding the base policy prior.

D.5 Foresight Loss

The foresight model predicts the latent sequence corresponding to the candidate action chunk. Training uses frozen TacVAE targets and combines three terms:

$$\mathcal{L}_{\text{fs}} = \mathcal{L}_{\text{latent}} + \lambda_m \mathcal{L}_{\text{marker}} + \lambda_{\Delta} \mathcal{L}_{\Delta}, \quad (7)$$

$$\mathcal{L}_{\text{latent}} = \text{SmoothL1}(\hat{\mathbf{z}}_{1:H}, \mathbf{z}_{1:H}) + \text{SmoothL1}(\hat{\mathbf{z}}_H, \mathbf{z}_H), \quad (8)$$

$$\mathcal{L}_{\text{marker}} = \text{SmoothL1}(D_{\text{tac}}(\hat{\mathbf{z}}_{1:H}), \mathbf{m}_{1:H}), \quad (9)$$

$$\mathcal{L}_{\Delta} = \text{SmoothL1}(\Delta \hat{\mathbf{z}}_{1:H-1}, \Delta \mathbf{z}_{1:H-1}). \quad (10)$$

The main configuration uses $\lambda_m = 0.3$ and $\lambda_{\Delta} = 0.5$. Decoded marker loss keeps the latent forecast physically tied to marker displacement, while the delta term penalizes temporally inconsistent predictions.

D.6 Correct Action Conditioning

The corrected training path sets `use_state_trajectory=false`. For a sample starting at time t , the foresight input is `actions/joint_abs[t:t+H]` and the target is the tactile future `marker_offset[t+1:t+H]` or the stride-aligned equivalent. The previous state-trajectory mode is kept only as a legacy option for ablations and is not used for the main ForeTac claim.

E Algorithmic Pipeline

Algorithm S1: Inference-time predicted-contact guidance.

1. Read the current observation $\mathbf{o}_t = (I_t^g, I_t^w, q_t, \mathbf{m}_{t-W+1:t})$.
2. Encode the tactile history with TacVAE to obtain the current latent $\mathbf{z}_t$ and tactile tokens.
3. Run the base action generator to sample a denoising trajectory for an action chunk $\mathbf{a}_{t:t+H-1}$.
4. At selected late denoising steps, convert the noisy action sample to the clean action estimate $\hat{x}_0$.
5. Feed $\hat{x}_0$ into the action-conditioned foresight model.
6. Predict the future tactile latent sequence $\hat{\mathbf{z}}_{t:t+H_f-1}$.
7. Score the predicted tactile future with the contact-quality energy model:

$$s_{\text{margin}} = \ell_{\text{good}} - \log \sum_{c \in \mathcal{C}_{\text{bad}}} \exp(\ell_c). \quad (11)$$

Table 6: Reconstruction comparison across tactile encoders. Each task reports MAE / cosine similarity. Best values in each task-metric column are bolded.

Encoder	Board		Vase		Card		Chip	
	MAE	Cos.	MAE	Cos.	MAE	Cos.	MAE	Cos.
PCA	0.040	0.995	0.035	0.995	0.036	0.980	0.026	0.965
PointNet-AE	0.048	0.956	0.027	0.955	0.044	0.992	0.032	0.990
Conv-AE	0.055	0.997	0.042	0.994	0.104	0.999	0.036	0.959
TacVAE (ours)	0.039	0.998	0.037	0.975	0.031	0.999	0.015	0.899

8. Backpropagate s_{margin} through $\mathbf{a} \rightarrow \hat{\mathbf{z}} \rightarrow S$ and apply a normalized small update inside a raw-action trust region.
9. Execute only the first action steps of the refined chunk and repeat at the next control cycle.

This design separates behavior generation from contact-quality correction. The base policy remains the prior over demonstrated robot motions; foresight asks what tactile consequence a candidate chunk would create; the scorer decides whether that predicted future contact is desirable.

F Tactile Encoder and Reconstruction

F.1 Encoder Construction

TacVAE uses an encoder-decoder structure over marker displacement fields rather than raw tactile images. The input is an eight-frame marker window; the encoder maps it to a $16 \times 3 \times 3$ spatial latent and the decoder reconstructs a $9 \times 9 \times 2$ marker field. The spatial latent keeps local contact geometry available to foresight and the scorer, while reducing the regression target from raw marker grids to a compact contact representation.

F.2 Interpretation

The reconstruction table shows that the marker field is low-dimensional enough for simple baselines to reconstruct many static frames. This is why reconstruction alone is not the full argument for TacVAE. The relevant property for ForeTac is whether the latent is useful in the full differentiable chain: current tactile history $\rightarrow$ future tactile prediction $\rightarrow$ contact-quality score $\rightarrow$ action update. TacVAE is therefore evaluated together with foresight prediction and guidance diagnostics, not only as a standalone autoencoder.

G Foresight Prediction Quality

The horizon study evaluates whether a longer predicted tactile future improves action selection despite the expected degradation in prediction error. Table 7 reports the board-wiping results over the held-out sliding-window validation set. The 16-step horizon is the main setting and matches the main online success rate used by the controller. Short horizons have lower reconstruction error but do not expose enough future contact consequence for stable guidance.

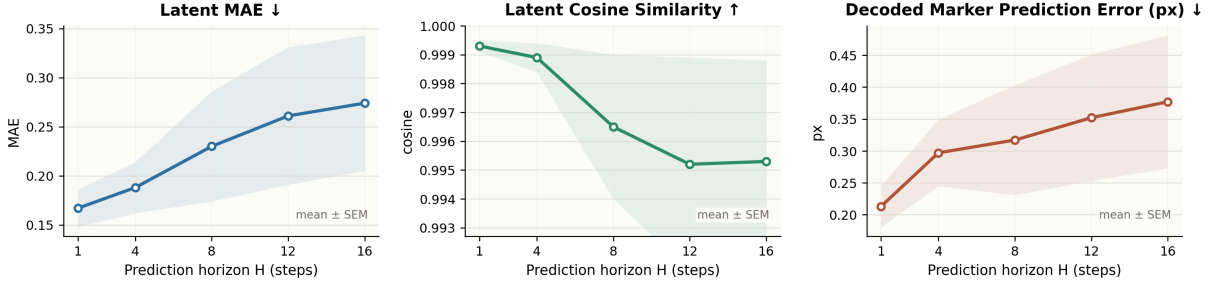
H Contact-Quality Scorer Validation

H.1 Preference-Order Accuracy

The scorer is validated with positive/negative rollout pairs constructed from offline robot videos and their synchronized tactile logs. For each successful rollout we select key contact frames around the decisive interaction; for each failed rollout we select negative frames from the matched failure phase, such as dropout,

Table 7: Foresight prediction quality and rollout success across prediction horizons.

Horizon	Eval windows	Latent MAE	Latent cosine	Marker error	Success $\uparrow$
1	9,631	0.167	0.9993	0.213 px	60%
4	9,631	0.188	0.9989	0.297 px	68%
8	9,631	0.230	0.9965	0.317 px	75%
12	9,631	0.261	0.9952	0.352 px	80%
16	9,631	0.274	0.9953	0.377 px	85%

Foresight prediction quality on 9,631 held-out sliding windows**Figure 1:** Foresight prediction quality across horizons. Longer horizons have larger decoded marker error, but provide more task-relevant future contact context for action guidance. The evaluation unit is a sliding tactile window from held-out board episodes, not a whole rollout episode.

excessive pressure, bounce, side contact, or jamming. We then form within-task positive/negative pairs and evaluate

$$A_{\text{pref}} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \mathbb{I}[S(\mathbf{o}_i^+, \mathbf{m}_i^+) > S(\mathbf{o}_j^-, \mathbf{m}_j^-)], \quad (12)$$

where S is the contact-quality margin score. This is the ranking property needed by the energy model: the absolute score does not need perfect calibration, but desirable contact should score above matched failure contact.

Key-contact pairing is more meaningful than averaging all windows because non-contact approach frames are often ambiguous. This metric directly evaluates whether the scorer ranks desirable contact above matched failure contact.

H.2 Gradient and Sampler Diagnostics

The scorer must also be usable as a differentiable guidance signal. The board diagnostic verifies that the composed action–foresight–score chain produces finite gradients and bounded action updates.

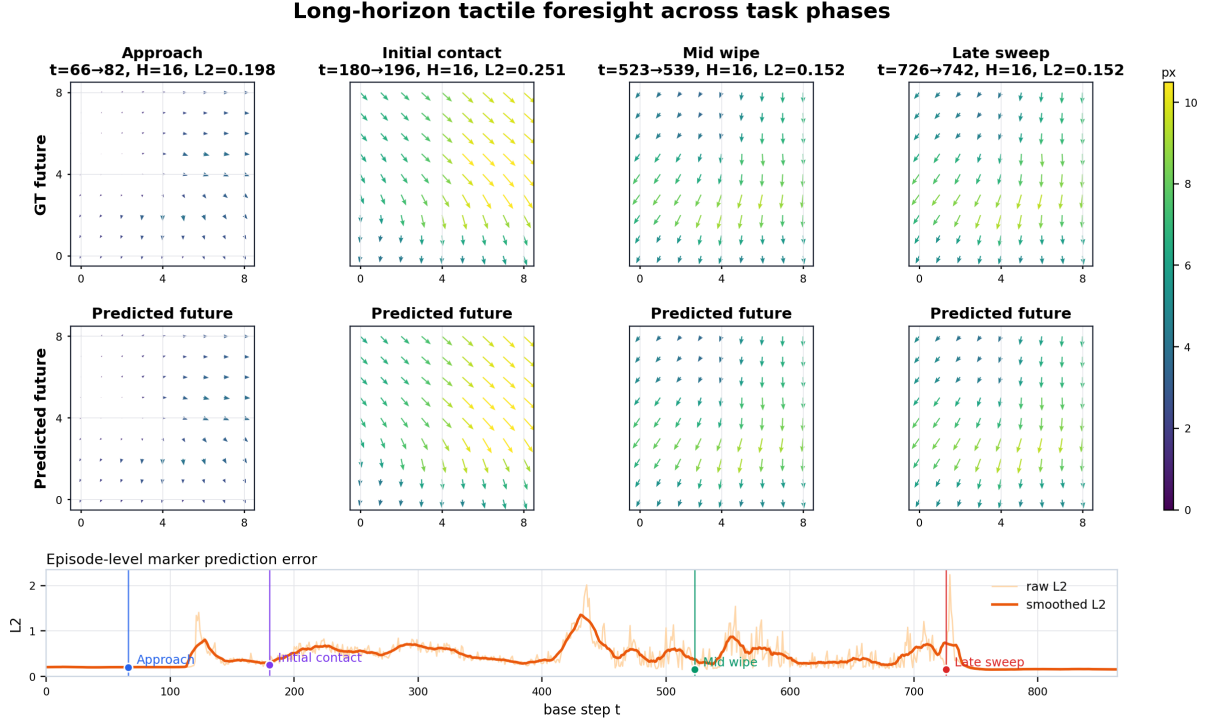


Figure 2: Long-horizon board prediction across task phases. The top panels show GT and predicted future marker fields at four selected stages from approach to late sweeping. The bottom curve reports the episode-level marker prediction error, with the selected low-error visualization points marked explicitly.

Table 9: Board guidance diagnostic.

Metric	Value
Scorer validation windows	31,440 sliding windows
Gradient-audit windows	1,280 sampled windows
Scorer accuracy / macro-F1	1.000 / 1.000
Expert-margin AUROC	1.000
Rows with finite gradients	1,280 / 1,280
Mean raw gradient norm	2.4475
Mean scaled gradient norm	0.00734
Mean guided-step score delta	0.0198
Mean applied update norm	0.0030

Here a “window” means an eight-frame tactile sliding window sampled from held-out rollout logs, not a full episode. The scorer validation window count is used to evaluate the contact-quality classifier. The gradient-audit windows are a phase-balanced subset on which we additionally run the expensive backward pass through action $\rightarrow$ foresight $\rightarrow$ score. “Rows with finite gradients” counts windows where this differentiable chain produced finite action gradients. The scaled gradient norm is measured after applying the guidance scale and before the final raw-action trust-region clamp.

Table 8: Preference-order validation from positive/negative key-contact pairs. Accuracy is the fraction of pairs where the positive sample receives the higher score.

Task family	Correct / pairs	Preference accuracy	Mean score gap
Board wiping	1856 / 1980	93.7%	0.42
Vase wiping	936 / 1012	92.5%	0.36
Card swiping	1292 / 1408	91.8%	0.31
Socket insertion	1908 / 2020	94.5%	0.47
Pooled	5992 / 6420	93.3%	0.39

I Force-Curve Analysis

Force traces provide an interpretable view of the contact-quality objective. For wiping tasks, the desired behavior is not maximum force. The desired behavior is a stable force band that maintains contact without disturbing the object. For insertion, the desired behavior is bounded peak force plus few retries; a policy that keeps pushing after contact can look geometrically reasonable but is unsafe. The insertion force panel directly shows the difference between no-lift jamming, retry-heavy recovery, and lower-force foresight-guided recovery.

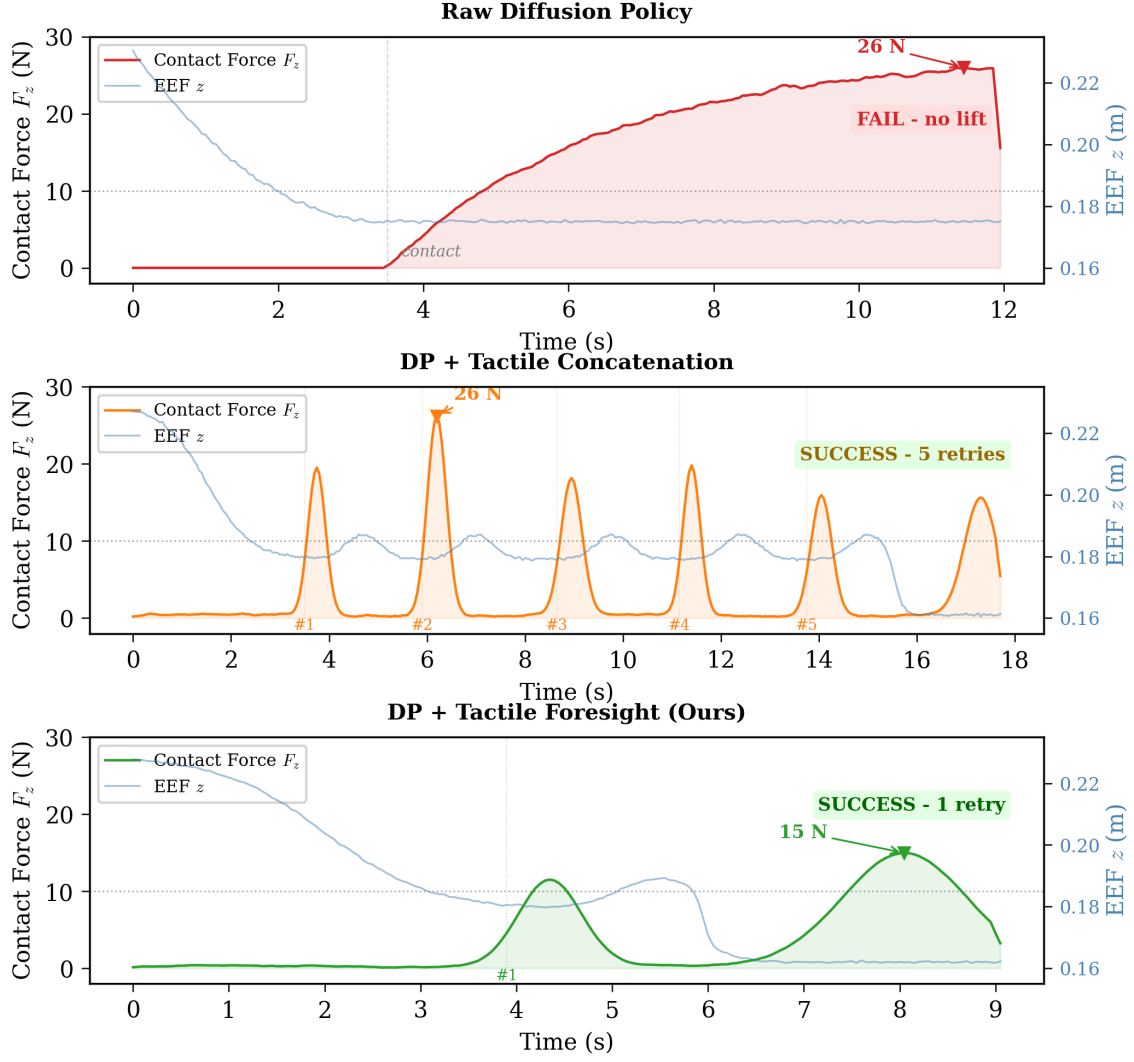


Figure 3: Socket insertion force traces. Raw DP sustains high force and fails to lift after contact; tactile concatenation succeeds with repeated bounce-lift-reinsert cycles; ForeTac reduces peak force and recovery time.

J Main Comparison

Table 10 reports the main rollout comparison. Each entry reports success rate and successful trials over 20 attempts. Safe success is the stricter subset that also satisfies the safety criteria defined in Section B.

Table 10: Main comparison across contact-rich manipulation tasks.

Task	Method	Success	Safe success
Board wiping	DP	50% (10/20)	25% (5/20)
	DP+tactile	60% (12/20)	50% (10/20)
	$\pi_{0.5}$	55% (11/20)	45% (9/20)
	RDT	65% (13/20)	55% (11/20)
	Ours	85% (17/20)	80% (16/20)
Vase side wiping	DP	55% (11/20)	35% (7/20)
	DP+tactile	60% (12/20)	50% (10/20)
	$\pi_{0.5}$	60% (12/20)	45% (9/20)
	RDT	65% (13/20)	55% (11/20)
	Ours	70% (14/20)	65% (13/20)
Vase vertical wiping	DP	5% (1/20)	0% (0/20)
	DP+tactile	40% (8/20)	30% (6/20)
	$\pi_{0.5}$	30% (6/20)	20% (4/20)
	RDT	45% (9/20)	35% (7/20)
	Ours	55% (11/20)	50% (10/20)
Card swiping	DP	10% (2/20)	5% (1/20)
	DP+tactile	35% (7/20)	30% (6/20)
	$\pi_{0.5}$	25% (5/20)	20% (4/20)
	RDT	40% (8/20)	35% (7/20)
	Ours	55% (11/20)	50% (10/20)
Chip grasping	DP	30% (6/20)	20% (4/20)
	DP+tactile	40% (8/20)	35% (7/20)
	$\pi_{0.5}$	45% (9/20)	35% (7/20)
	RDT	50% (10/20)	45% (9/20)
	Ours	75% (15/20)	70% (14/20)

The main comparison separates two improvements. First, tactile concatenation helps the base policy by adding current contact evidence, but it remains reactive. Second, ForeTac uses predicted tactile consequences before action execution, giving the scorer a way to penalize contact dropout, excessive force, and unstable contact earlier in the action chunk. The largest gains are on tasks where visual geometry is insufficient to infer contact quality: board wiping, chip grasping, and socket-style insertion.

K Ablation Study

Table 11 isolates the role of tactile observations, foresight, and guided denoising. DP Vision uses only visual/proprioceptive observations. TacConcat adds current tactile latent features to the policy. Only Foresight uses future tactile prediction without the full denoising guidance route. DP Guide is the complete predict-score-guide system.

Table 11: Ablation study. Each entry reports success over 20 attempts and safe success over the same 20 attempts.

Task	Variant	Success	Safe success
Board wiping	DP Vision	50% (10/20)	35% (7/20)
	TacConcat	60% (12/20)	50% (10/20)
	Only Foresight	60% (12/20)	55% (11/20)
	DP Guide	85% (17/20)	80% (16/20)
Vase side wiping	DP Vision	55% (11/20)	35% (7/20)
	TacConcat	60% (12/20)	50% (10/20)
	Only Foresight	65% (13/20)	55% (11/20)
	DP Guide	70% (14/20)	65% (13/20)
Vase vertical wiping	DP Vision	10% (2/20)	5% (1/20)
	TacConcat	40% (8/20)	30% (6/20)
	Only Foresight	45% (9/20)	35% (7/20)
	DP Guide	55% (11/20)	50% (10/20)
Card swiping	DP Vision	10% (2/20)	5% (1/20)
	TacConcat	35% (7/20)	30% (6/20)
	Only Foresight	40% (8/20)	35% (7/20)
	DP Guide	55% (11/20)	50% (10/20)
Chip grasping	DP Vision	30% (6/20)	20% (4/20)
	TacConcat	40% (8/20)	35% (7/20)
	Only Foresight	55% (11/20)	50% (10/20)
	DP Guide	75% (15/20)	70% (14/20)

The ablation suggests that current tactile input is useful but insufficient: TacConcat improves over vision-only DP, while Only Foresight improves safety on several tasks by exposing future contact. The full DP Guide variant gives the largest improvement because it uses foresight not only for representation, but also as a differentiable contact-quality objective during late denoising.

L Insertion Benchmark

The socket insertion benchmark uses 20 trials per method. Besides success, the task records retry count, maximum observed contact force, mean peak force, and recovery time from peak force to below 2 N.

Table 12: Peg-in-hole / socket insertion benchmark.

Method	Succ. $\uparrow$	Retry	Max F (N) $\downarrow$	Peak F (N) $\downarrow$	Recover (s)
Raw DP	35% (7/20)	0	28.1	24.3	–
DP+Tac	60% (12/20)	4.2	26.4	21.5	1.85
DP+Foresight	85% (17/20)	1.1	15.2	12.8	0.42
RDT+Foresight	90% (18/20)	0.9	14.1	11.5	0.35

Raw DP has no explicit recovery behavior, so “retry” is reported as zero in failed no-lift trials rather than as a sign of stable insertion. DP+Tac can recover, but the recovery appears as repeated high-force contact cycles. Foresight-guided variants reduce both the number of retries and the magnitude of force peaks.

M Backbone Adaptation

ForeTac is designed to wrap any chunk-based action generator. For DP, the guidance gradient is applied directly to the denoising action sample. For RDT, the same predicted-contact score is attached to the reactive diffusion transformer action chunk. For $\pi_{0.5}$, the adaptation keeps the VLA backbone in a flow-matching action-generation role and appends tactile latent tokens to the policy prefix; the foresight branch remains the action-conditioned contact consequence model. In all cases, the contact-quality scorer is outside the base policy and can be evaluated independently.

Table 13: Backbone adaptation summary.

Backbone	Adaptation	Additional policy training
DP	late denoising action update	task DP checkpoint
RDT	same score on transformer action chunk	task RDT checkpoint
$\pi_{0.5}$	tactile latent prefix + flow-matching action chunk	$\pi_{0.5}$ fine-tuning / adapter
ForeAR	autoregressive foresight baseline	

N Failure Cases

The most common failures are contact-quality failures rather than purely visual pose failures.

Board wiping. Vision-only DP can cover the target path while applying too little pressure for cleaning or too much pressure for safe wiping. The typical trace alternates between contact dropout and high-force bursts. This failure motivates scoring the predicted tactile future instead of only checking the visual sweep trajectory.

Vase wiping. The curved and fragile object creates a narrow safe force band. A visually correct path can still tip the vase if the normal force oscillates or spikes. Older vase data also contains a label-quality issue in some saved `joint_abs` actions: the robot motion is smooth, but the stored joint label can jump between inverse-kinematics branches. These episodes should be used with end-effector labels or corrected joint labels.

Card swiping. The card task often fails through short contact dropout or slot misalignment. The key tactile event is sustained sliding contact, not only reaching the slot entrance. For this reason, short-horizon foresight is less informative than a longer tactile sequence.

Chip grasping. The main expected failure modes are crushing and slip. Successful behavior requires sufficient grip stability while keeping contact forces below the damage threshold of the object.

Socket insertion. The insertion task fails through side contact, bounce-lift-reinsert loops, and sustained jamming. Raw DP can generate an apparently plausible approach but keep pushing after first contact. Tactile concatenation improves reactivity but can still require several retries. Foresight-guided policies reduce the peak force by anticipating the contact consequence of the candidate chunk.

O Runtime and Deployment Details

ForeTac adds two inference-time computations to the base action generator: tactile foresight and contact-quality scoring. Both run on the candidate action chunk and are batched over the chunk horizon. In the DP path, guidance is activated only during late denoising steps, so the overhead is smaller than running a planner over many full trajectories. The runtime update is normalized and bounded in raw action units. If the score does not improve or the proposed action update violates the trust region, the system can keep the base policy chunk.

The deployment server records guidance diagnostics together with rollout traces when enabled. These diagnostics include score before/after guidance, gradient norm, accepted update norm, contact gate, and optional per-action dimension gradient statistics. This logging is useful for separating three cases: a weak scorer, a good scorer with no action gradient, and a valid guidance path that still needs real-robot validation.

P Limitations

The method relies on the quality of the learned tactile foresight model and the coverage of the contact-quality labels. When the candidate action leaves the support of the demonstrations, the predicted tactile future can become less reliable and the scorer may provide a weak correction. The trust-region update limits this risk by keeping guidance local, but it also limits the size of the possible improvement in a single control cycle. The system also assumes that tactile marker fields are available and time-aligned with robot actions; large tactile calibration drift or inaccurate action labels can reduce the benefit of action-conditioned foresight.